Advanced Programming Techniques

fbeit
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Question https://poll.mnl.de (Code:)

- **Constructors ...**
  1. Must assign values to all members of an object
  2. Must assign values to all data members of an object
  3. Must assign values to most data members of an object
  4. Should assign values to all data members of an object
  5. Should assign values to most data members of an object

Question https://poll.mnl.de (Code:)

- **Which function can never change the value of a variable which is defined in its invoker's context?**

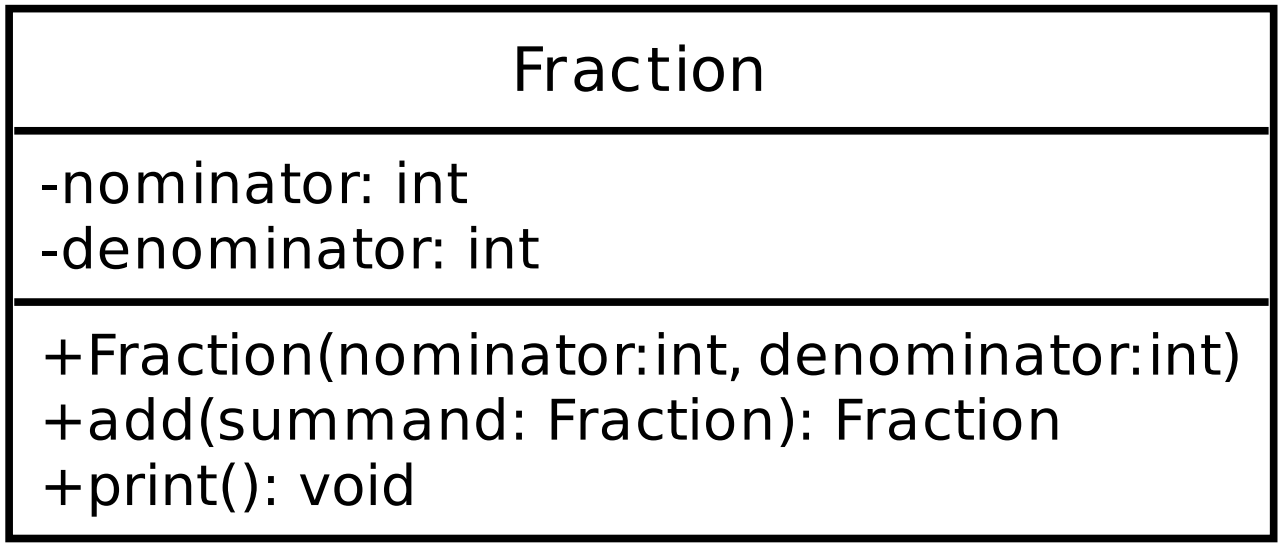
1. `void f (int* a, long b);`
2. `int f (double a, long b);`
3. `double f (int& a, double b);`
4. `float& f (int a, long* b);`

Operator Overloading

fbeit
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Simple class: Fraction

- **Numbers like 1/3 cannot be represented precisely**
  - 0,3333333333... will eventually be cut off, even if you use e.g. `double`
- **Own (user defined) type `Fraction`**

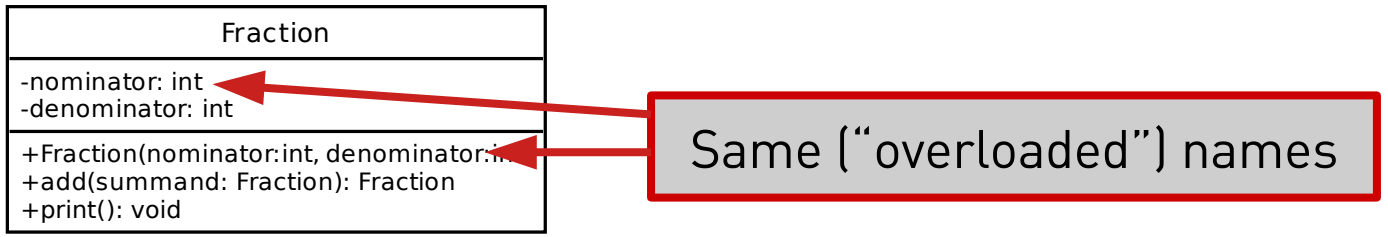


Class implementation in Eclipse

Implementation caveat: overloaded names

- **Sometimes "natural" (immediately coming to mind) attribute names and parameter names are the same**
  - Parameter name (defined in nested scope) overloads ("shadows") attribute name
  - Use **`this`** to reference attribute
  - **`this`** is (within method implementations) a pointer to the object that the method is invoked for

```
Fraction::Fraction(int nominator,
        int denominator) {
    this->nominator = nominator;
    this->denominator = denominator;
}
```



Make result look nicer: gcd

- **Results from add can be simplified by reducing the fraction**
  - Find greatest common denominator (gcd)
  - Divide nominator and denominator by gcd
- **For finding gcd, ask mathematicians (or Wikipedia)**
- **Algorithm (Pseudo code):**

```
function gcd(a, b)
  if b = 0
    return a;
  else
    return gcd(b, a mod b);
```

Recursive function (calls itself)

Private methods

- **Private methods are not part of a class' "interface"**
  - Can only be accessed (called) from other member functions (just like private attributes can only be accessed from member functions)
  - Cannot be called from "outside", i. e. the method is invisible to the user of the object
  - Perfect for helper method `gcd`
    - Used "internally" to simplify result
    - Obviously not part of `Fraction`'s interface (nobody would expect a fraction to expose a method for calculating the gcd, it's not part of the behavior expected from a fraction)

Using `Fraction`

- Simple test:

```
Fraction a{1,2};
Fraction b{1,4};
a.add(b).print();
```

- Yes, but…
  - Usually we use "+" for adding numbers ("a+b") (→ Test with `Fraction`)
  - Wouldn't it be nice if we could use this for our `Fractions` as well?
  - C++ supports defining how operators work with classes
  - Called: operator overloading

C++ handling of operations

- **Operations are "internally rewritten"**
  - Let
    - "@" be an arbitrary operator and
    - "`type1 a; type2 b;`" two variables,
  - then
    - "`a @ b`" is "internally rewritten to: "`operator@(a, b)`"
      (e.g. "`a + b`" becomes "`operator+(a, b)`")
  - Functions "`operator@(type1 a, type2 b)`" are well-known (pre-defined) for the built-in (primitive) types `int`, `long`, `double` etc. and the operators '`+`', '`-`', '`*`', '`/`', ...
  - They can additionally be defined for classes (user defined types)

Function for overloading an operator

- **Note that this is a function (outside class definition)**
  - Declaration (`fraction.h`):

```
Fraction operator+(Fraction leftSummand, Fraction rightSummand);
```

  - Implementation (`fraction.cpp`)

```
Fraction operator+(Fraction leftSummand,
    Fraction rightSummand) {
    return leftSummand.add(rightSummand);
}
```

- **Usage**

```
Fraction a{1,2};
Fraction b{1,4};
(a + b).print();
```

Eclipse Demo

Improved `print()`

- Still not nice: **`print()`**
- C++ style: "**`int i; cout << i << endl;`**"
- Can we do: "**`Fraction a; cout << a << endl;`**"?
- **Two steps:**
  - "`<<`" is a left associative operator (https://en.wikipedia.org/wiki/Operator_associativity)

    → `(cout << a) << endl;`
  - Overload "`<<`" to accept `ostream` and `Fraction` and return `ostream`

    →
    ```
    std::ostream& operator<<(
        std::ostream& leftOperand, Fraction rightOperand);
    ```
    - Left operand (`ostream`) is passed in and out by reference (i.e. always same object)

Eclipse Demo

Achieved result

- **From old to new:**

```
Fraction a{1,2};
Fraction b{1,4};
a.add(b).print();
```

  - becomes:

```
cout << a + b << endl; // Much easier to read!
```

  - … which is “Rewritten” by compiler (never write this!) to:

```
operator<< (operator<< (cout, operator+(a,b)), endl);
```

- **Note that overloading does not change operator precedence**
  (see https://en.cppreference.com/w/cpp/language/operator_precedence)

Using `friend` access

- When writing functions for operator overloading, private members of operands are not accessible
- Sometimes access should be granted to non-member function
- Can be done with "**`friend`**" declaration:

```
class C {
    // ..
    friend void f(C param);
};
```

  - Grants function f access to private members of C

Alternative: Overloading with method

- **Operator "rewriting" (added rules if first operand is class)**
  - Let
    - "@" be an arbitrary operator and
    - "`type1 a; type2 b;`" two variables, where `type1` is a class
  - then
    - "`a @ b`" is "internally rewritten to: "`a.operator@(b)`", and only
    - if this method does not exist, it is "internally rewritten to: "`operator@(a, b)`"
  - e.g. "`a + b`" may become
    - "`a.operator+(a)`" or (if this method is not known)
    - "`operator+(a, b)`"

Unary operators work as well

- **Examples**
  - Overload pre-/postincrements
    - Special case: dummy parameter to make post-increment recognizable
- **In general, make things work similar to "`int`"**
- **Use proper return type**
  - Return type of all assignment operators ("`=`", "`+=`", "`-=`", ...) is the variable, not its value! ("`T& operator=(T const& orig)`"), e.g. "`(a+=5) = 6`" is valid (though not very useful)
- **Don't invent "exotic" (non intuitive) usages**

Eclipse Demo

When to use functions or methods

- **Overloading with functions**
  - must be used is the left operand is not defined in your code, e.g.
    - "`cout << ...`" → Cannot add method to `cout` (`ostream`), or
    - "`int i; Fraction f; i * f;`"
- **Overloading with method**
  - must be used for `operator=`
  - should be preferred when access to private members is needed